

Binary Search Tree

@author Yangxin Wu

Last edited: 2025.06.09

- [Binary Search Tree \(BST\)](#)
- [Self-balancing BST](#)
 - [Rotate](#)
 - [AVL\(Adelson-Velsky and Landis\) Tree](#)
 - [B\(Bayer\) Tree](#)
 - [B+ Tree](#)

🔧 有用的小工具: [Data Structure Visualization](#).

👏 感谢 *The University of San Francisco*, 制作了这个能够可视化数据结构操作过程的工具, 帮助用户更好的理解数据结构。

📄 可以参考一篇写的很好 [Blog](#) by 知乎.勤劳的小手 和 [Wikipedia](#)。

Binary Search Tree (BST)

BST 是一种二叉树的树形数据结构, 其定义如下:

1. 空树是二叉搜索树。
2. 若二叉搜索树的左子树不为空, 则其左子树上所有点的权值均小于其根节点的权值。
3. 若二叉搜索树的右子树不为空, 则其右子树上所有点的权值均大于其根节点的权值。
4. 二叉搜索树的左右子树均为二叉搜索树。

我们可以定义 BST 中的一个节点如下 (我们默认权值为整数):

```
struct TreeNode {
    int key;
    TreeNode* left;    // 左子树
    TreeNode* right;   // 右子树
    int size;          // 当前节点为根的子树大小
    int count;         // 当前节点的重复数量
};

TreeNode* initNode(int value) {
    TreeNode new_node = (TreeNode*)malloc(sizeof(TreeNode));

    new_node->key = value;
    new_node->left = NULL;
    new_node->right = NULL;
    new_node->size = 1;
    new_node->count = 1;
}
```

```
    return new_node;
}
```

遍历：由 **BST** 的递归定义可得，**BST** 的中序遍历生成非降的权值序列，时间复杂度为 $O(n)$ 。

```
void LTRTraversal(TreeNode* root) {
    if (root == NULL) return;

    LTRTraversal(root->left);
    printf("%d, ", root.key);
    LTRTraversal(root->right);
}
```

取最大值/最小值：**BST** 上的最小值为 **BST** 左链的顶点，最大值为 **BST** 右链的顶点。（吉列的往左或往右即可）

```
int getMin(TreeNode* root) {
    if (root == NULL) return -1;

    while (root->left != NULL) root = root->left;

    return root->key;
}

int getMax(TreeNode* root) {
    if (root == NULL) return -1;

    while (root->right != NULL) root = root->right;

    return root->key;
}
```

搜索：类比对分查找即可，比如在以 `root` 为根节点的 **BST** 中搜索一个值为 `value` 的节点。分类讨论如下：

- 若 `root` 为空，返回 `false`。
- 若 `root` 的权值等于 `value`，返回 `true`。
- 若 `root` 的权值大于 `value`，在 `root` 的左子树中继续搜索。
- 若 `root` 的权值小于 `value`，在 `root` 的右子树中继续搜索。

复杂度分析：对于一个节点数为 n 的 **BST**，随机构造这样一棵 **BST** 的期望高度和搜索的最优时间复杂度为 $O(\log n)$ (与对分查找一致)，最坏为 $O(n)$ (当 **BST** 退化为线性表时)。

```
bool search(TreeNode* root, int target) {
    if (root == NULL) return false;
```

```

    if (root->key == target) {
        return true;
    } else if (target < root->key) {
        return search(root->left, target);
    } else {
        return search(root->right, target);
    }
}

```

插入节点：在以 `root` 为根节点的 **BST** 中插入一个权值为 `value` 的节点。
分类讨论如下：

- 若 `root` 为空，直接返回一个值为 `value` 的新节点。
 - 若 `root` 的权值等于 `value`，该节点的附加域该值出现的次数自增 1。
 - 若 `root` 的权值大于 `value`，在 `root` 的左子树中插入权值为 `value` 的节点。
 - 若 `root` 的权值小于 `value`，在 `root` 的右子树中插入权值为 `value` 的节点。
- 复杂度分析：显然与搜索一致。

```

TreeNode* insert(TreeNode* root, int value) {
    if (root == NULL) return initNode(value);

    if (value < root->key) {
        root->left = insert(root->left, value);
    } else if (value > root->key) {
        root->right = insert(root->right, value);
    } else {
        root->count++; // 节点值相等，增加重复数量
    }

    // 维护size
    int left_child_size = root->left ? root->left->size : 0;
    int right_child_size = root->right ? root->right->size : 0;
    root->size = root->count + left_child_size + right_child_size;

    return root;
}

```

删除节点：在以 `root` 为根节点的 **BST** 中删除一个值为 `value` 的节点。
分类讨论如下：

- 若 `root` 为空，直接返回 `root`。
- 若 `root` 的权值小于 `value`，在 `root` 的右子树中删除权值为 `value` 的节点。
- 若 `root` 的权值大于 `value`，在 `root` 的左子树中删除权值为 `value` 的节点。
- 若 `root` 的权值等于 `value`，该节点的附加域该值出现的次数自增 1。

```

// 辅助函数：找到权值最小的节点
TreeNode* _findMinNode(TreeNode* root) {
    while (root->left != NULL) {
        root = root->left;
    }
    return root;
}

TreeNode* remove(TreeNode* root, int value) {
    if (root == NULL) {
        printf("Invalid operation: target node doesn't exist.");
        return root;
    }

    if (value < root->key) {
        root->left = remove(root->left, value);
    } else if (value > root->key) {
        root->right = remove(root->right, value);
    } else {
        // 节点重复数量大于1，减少重复数量即可
        if (root->count > 1) {
            root->count--;
        } else {
            if (root->left == NULL) {
                // 如果该节点只有右子树，直接把当前节点删除，把右子树接上即可
                TreeNode* temp = root->right;
                free(root);
                return temp;
            } else if (root->right == NULL) {
                // 该节点只有左子树，同上
                TreeNode* temp = root->left;
                free(root);
                return temp;
            } else {
                TreeNode* successor = _findMinNode(root->right);
                root->key = successor->key;
                root->count = successor->count; // 更新重复数量
                // 如果 successor->count > 1时，调用的删除函数只会减少重复数量。
                // 因此需要将 successor->count 重置为 1 再调用。
                successor->count = 1;
                root->right = remove(root->right, successor->key);
            }
        }
    }
}

int left_child_size = root->left ? root->left->size : 0;
int right_child_size = root->right ? root->right->size : 0;
root->size = root->count + left_child_size + right_child_size;

```

```
    return root;
}
```

Self-balancing BST

如果一棵树的高度为 h ，在最坏的情况下，查找一个关键字需要对比 h 次，查找时间复杂度（也称为平均查找长度 ASL, Average Search Length）不超过 $O(h)$ 。

我们不难发现一棵 BST 的高度应该满足 $\log n \leq h \leq n$ ，也就是说为了降低操作的时间复杂度，我们需要使 BST 的高度尽量向 $\log n$ 靠近。由此，我们引入平衡树，即通过一定操作维持树的高度（平衡性）来降低操作的复杂度。

在介绍 AVL tree、B tree 和 B+ tree 之前，我们先介绍四种基本的调整平衡过程。

Rotate

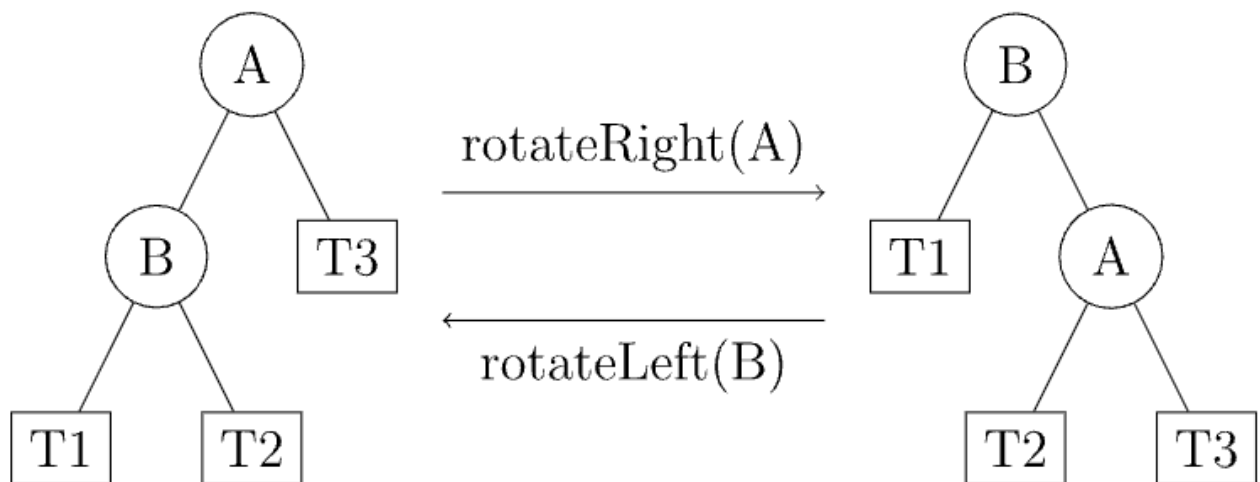
关于二叉平衡树，平衡的调整基本操作分为包括左旋 (Left Rotate) 和右旋 (Right Rotate) 两种。下面主要介绍左旋，右旋可通过镜像操作得到。

左旋也被称为「左单旋转」或「RR 平衡旋转」；右旋也被称为「右单旋转」或「LL 平衡旋转」。

以下图为例，

对于结点 B 的左旋操作为：将 B 的右孩子 A 向左上旋转，代替 B 成为根节点，将 B 结点向左下旋转成为 A 的左子树的根结点， A 的原来的左子树变为 B 的右子树。

对于结点 A 的左旋操作为：将 A 的左孩子 B 向右上旋转，代替 A 成为根节点，将 A 结点向右下旋转成为 B 的右子树的根结点， B 的原来的右子树变为 A 的左子树。

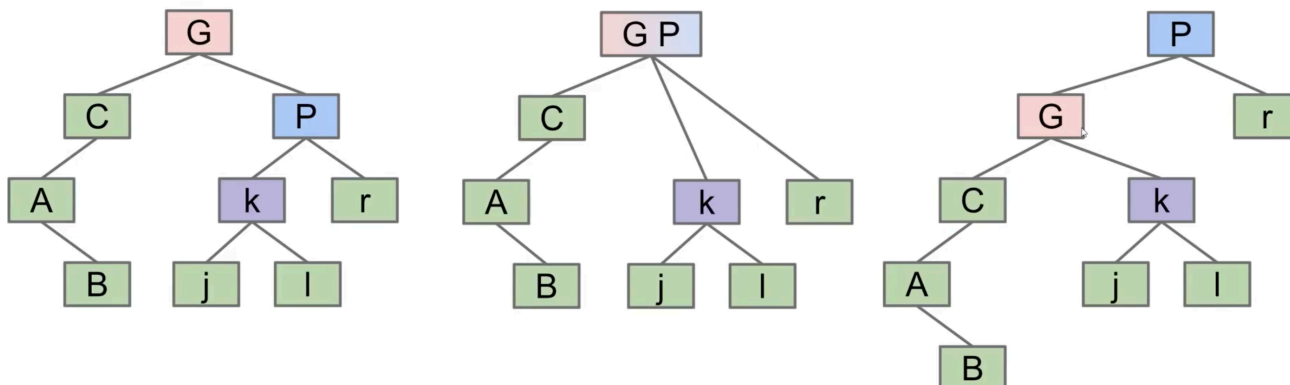


如果你觉得有点抽象，[CS61B.Sp21.Lecture18.RedBlackTrees](#) 中提到过另一种的对于 Rotate 过程的解释。我们可以将该过程分解为先向上调整 (merge) 再向下调整 (split) 的过

程:

rotateLeft(G): Let x be the right child of G. Make G the **new left child** of x.

- Can think of as temporarily merging G and P, then sending G down and **left**.
- Preserves search tree property. No change to semantics of tree.



由上，对于某一个结点，我们可以实现方法 rotateLeft 和 rotateRight 如下：

```
TreeNode* rotateLeft(TreeNode* root) {
    TreeNode* newRoot = root->right;
    root->right = newRoot->left;
    newRoot->left = root;

    // 更新相关节点的信息（不用关心具体函数实现）
    updateHeight(root);
    updateHeight(newRoot);

    return newRoot;
}

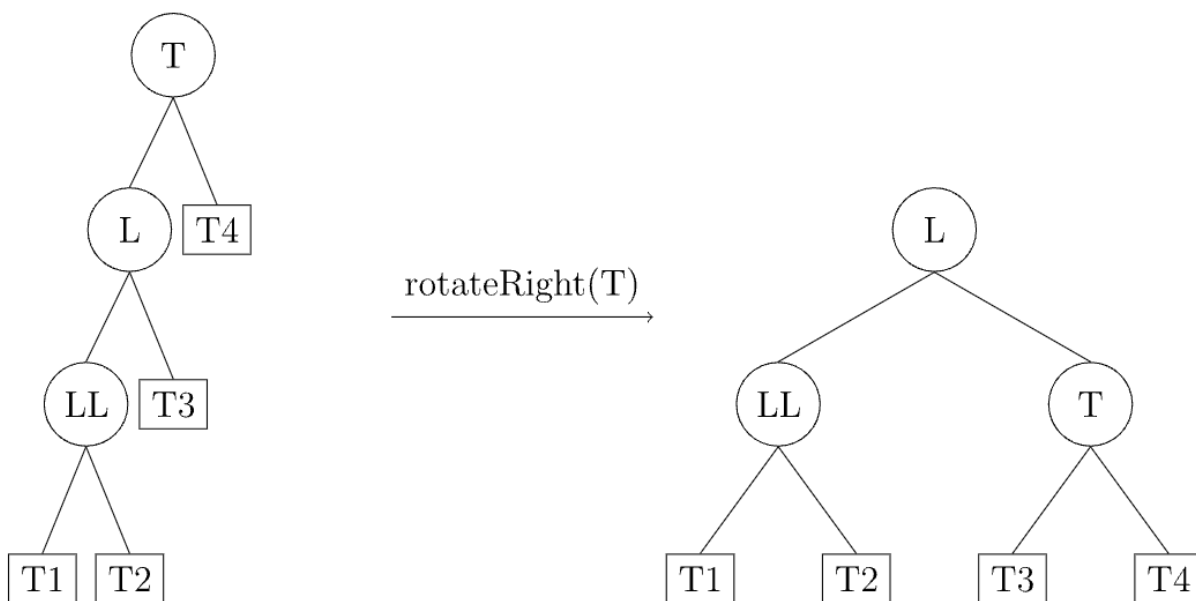
TreeNode* rotateRight(TreeNode* root) {
    TreeNode* newRoot = root->left;
    root->left = newRoot->right;
    newRoot->right = root;

    // 更新相关节点的信息（不用关心具体函数实现）
    updateHeight(root);
    updateHeight(newRoot);

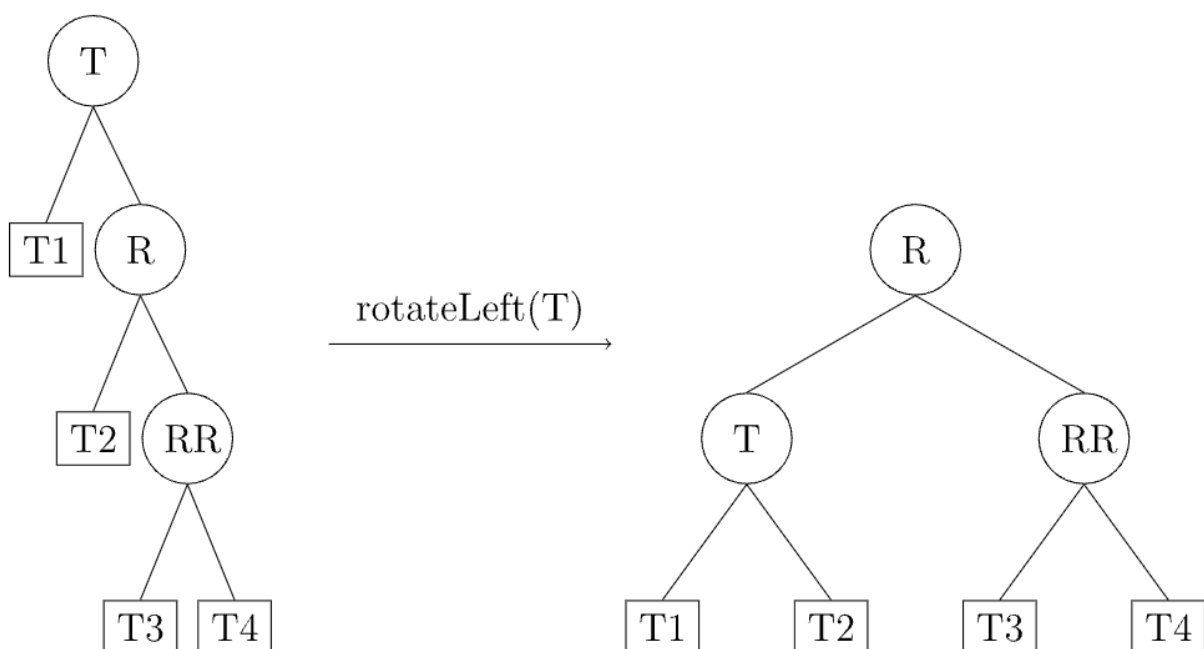
    return newRoot;
}
```

我们有四种平衡性破坏的情况：LL，RR，LR，RL。示意图来自 [OI Wiki](#)。

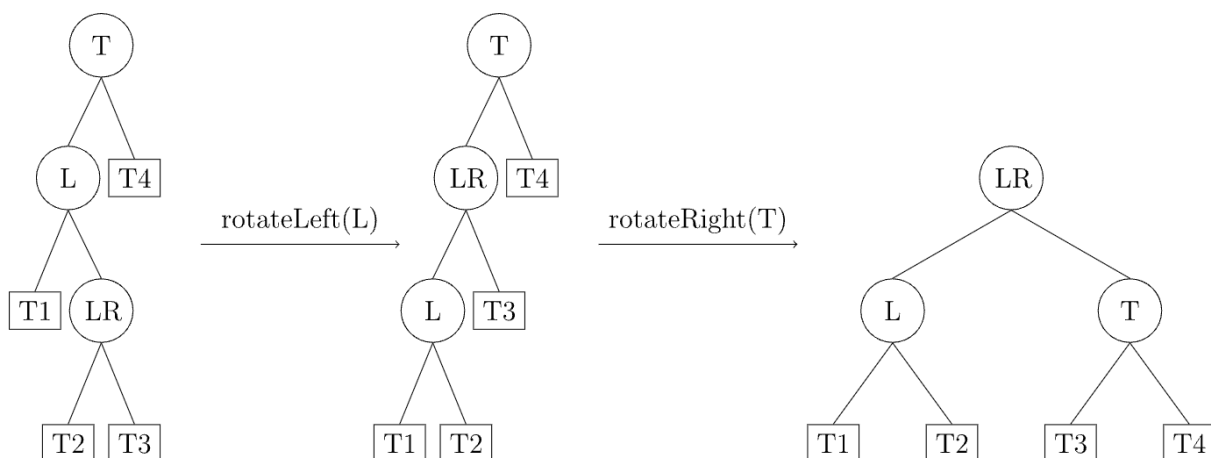
- LL: T 的左孩子的左子树过长导致平衡性破坏。
→ 右旋节点 T 。



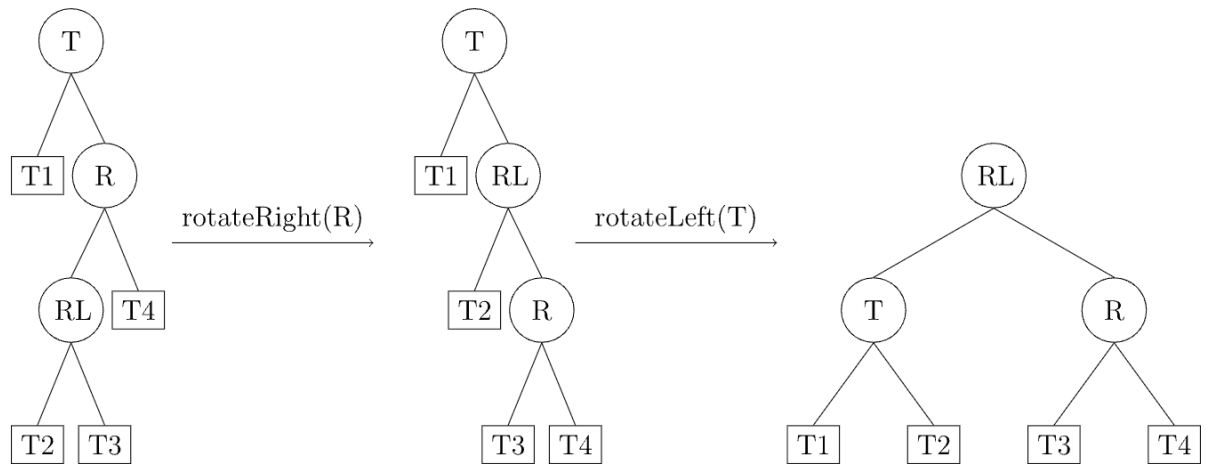
- **RR:** T 的右孩子的右子树过长导致平衡性破坏。
 → 左旋节点 T 。



- **LR:** T 的左孩子 L 的右子树过长导致平衡性破坏。
 → 先左旋节点 L ，成为 LL 型，再右旋节点 T 。



- **RL**: T 的右孩子 R 的左子树过长导致平衡性破坏。
→ 先右旋节点 R , 成为 **RR** 型, 再左旋节点 T 。



AVL(Adelson-Velsky and Landis) Tree

定义:

1. 空二叉树是一棵 **AVL** 树;
2. 如果 T 是一棵 **AVL** 树, 那么其左右子树也是 **AVL** 树, 并且 $|h(LT) - h(RT)| \leq 1$;
3. 平衡因子: 左子树高度 - 右子树高度

// 为了平衡因子的计算, 我们需要创建一个新的结构体, 继承TreeNode全部的属性, 并维护height变量。

```
struct AVLTreeNode {
    int key;
    TreeNode* left;    // 左子树
    TreeNode* right;   // 右子树
    int height;        // 以当前节点为根的树高
    int size;          // 当前节点为根的子树大小
    int count;         // 当前节点的重复数量
};
```

// 平衡因子计算

```
int getBalanceFactor(AVLTreeNode* root) {
    if (root == NULL)
        return 0;
    else
        balance_factor = x->left->height - x->right->height
    return balance_factor;
}
```

// 平衡性维护

```
AVLTreeNode* rebalance(AVLTreeNode* root) {
    int root_factor = getBalanceFactor(root);
```

```

    if(root_factor > 1 && getBalanceFactor(root->left) > 0) {
        // LL
        return rotateRight(root);
    } else if(root_factor > 1 && getBalanceFactor(root->left) <= 0) {
        // LR
        root->left = rotateLeft(root->left);
        return rotateRight(temp);
    } else if(root_factor < -1 && getBalanceFactor(root->right) <= 0) {
        // RR
        return rotateLeft(root);
    } else if((root_factor < -1 && getBalanceFactor(root->right) > 0) {
        // RL
        root->right = rotateRight(root->right);
        return rotateLeft(root);
    } else {
        // Nothing happened.
        return root;
    }
}

```

我们不难证明满足以上定义的 **BST** 高度为 $O(\log n)$.

Proof:

设 f_n 为高度为 n 的 **AVL** 树所包含的最少节点数，则有

$$f_n = \begin{cases} 1 & (n = 1) \\ 2 & (n = 2) \\ f_{n-1} + f_{n-2} + 1 & (n > 2) \end{cases}$$

因为 **AVL** 树需要满足根节点平衡因子的绝对值小于等于 1，那我们可以这样去构造一棵节点数最少的 **AVL** 树：使根节点的左子树为高度为 $n - 1$ 的 **AVL** 树，右子树为高度为 $n - 2$ 的 **AVL** 树。因此当 $n > 2$ 时， $f_n = f_{n-1} + f_{n-2} + 1$ 。

由于 $f_n + 1 = (f_{n-1} + 1) + (f_{n-2} + 1)$ ，我们不难发现 $\{f_n + 1\}$ 满足斐波那契数列的递推关系。根据斐波那契数列的通项公式可得：

$$\begin{aligned}
 f_n + 1 &= \frac{\sqrt{5}}{5} \left[\left(\frac{1 + \sqrt{5}}{2} \right)^{n+2} - \left(\frac{1 - \sqrt{5}}{2} \right)^{n+2} \right] \\
 \Rightarrow f_n &= \frac{5 + 2\sqrt{5}}{5} \left(\frac{1 + \sqrt{5}}{2} \right)^n + \frac{5 - 2\sqrt{5}}{5} \left(\frac{1 - \sqrt{5}}{2} \right)^n - 1
 \end{aligned}$$

斐波那契数列以指数的速度增长，对于树高 n 有：

$$n < \log_{\frac{1+\sqrt{5}}{2}}(f_n + 1) < \frac{3}{2} \log_2(f_n + 1)$$

因此 **AVL** 树的高度为 $O(\log f_n)$ ，这里的 f_n 为结点数。

插入/删除 操作

对于一组关键字输入序列，从一棵空树开始不断插入结点，最后构成**AVL**树。

每插入/删除一个结点后就应判断从该结点到根的路径上是否有结点发生不平衡（即左右子树是否满足定义 2），如有不平衡问题，利用进行树的调整使之平衡化（4 种平衡性被破坏的情况）。

在 [AVL Tree Visualization](#) 可以观察 AVL 树维护平衡的过程。

B(Bayer) Tree

定义一棵 m 阶 B 树如下：

1. 每个节点最多有 $m(m \geq 2)$ 个子节点，所有节点的关键字均按升序排列，并遵循左小右大原则。
2. 每一个除根节点外的非叶子节点最少有 $\lceil \frac{m}{2} \rceil$ 个子节点；如果根节点不是叶子节点，那么它至少有两个子节点。
3. 有 k 个子节点的非叶子节点拥有 $k - 1$ 个关键字。
4. 所有的叶子节点都在同一层。叶子节点除了包含关键字和关键字记录的指针外也有指向其子节点的指针，只不过其指针地址都为 NULL。

```
// 最小度 t（或者通常用 m 表示阶，t 对应 m/2）
// 每个非根节点至少有 t-1 个键，最多 2t-1 个键。
// 每个非根节点至少有 t 个子节点，最多 2t 个子节点。
// 假设最小度为 3，那么每个节点最多有 2*3-1 = 5 个键，最多有 2*3 = 6 个子节点。
#define T 3

// B树节点结构
typedef struct BTreeNode {
    int keys[2 * T - 1];           // 存储键的数组
    struct BTreeNode *children[2 * T]; // 存储子节点指针的数组
    int n;                         // 当前节点中键的数量
    bool isLeaf;                   // 是否为叶子节点
} BTreeNode;

// B树结构
typedef struct BTree {
    BTreeNode *root;
} BTree;

// 创建一个新节点
BTreeNode *createNode(bool isLeaf) {
    BTreeNode *newNode = (BTreeNode *)malloc(sizeof(BTreeNode));
    if (newNode == NULL) {
        perror("Failed to allocate memory for BTree node");
        exit(EXIT_FAILURE);
    }
    newNode->n = 0;
```

```

newNode->isLeaf = isLeaf;
for (int i = 0; i < 2 * T - 1; i++) {
    newNode->keys[i] = 0; // 初始化键为0
}
for (int i = 0; i < 2 * T; i++) {
    newNode->children[i] = NULL; // 初始化子节点指针为NULL
}
return newNode;
}

// 创建一棵新的B树
BTree *createBTree() {
    BTree *newTree = (BTree *)malloc(sizeof(BTree));
    if (newTree == NULL) {
        perror("Failed to allocate memory for BTree");
        exit(EXIT_FAILURE);
    }
    newTree->root = createNode(true); // 初始时根节点是叶子节点
    return newTree;
}

```

查找：

假设需要查找的是 k ，那么从根节点开始，从上到下递归的遍历树。在每一层上，搜索的范围被减小到包含了搜索值的子树中。子树值的范围被它的父节点的元素确定。

```

// 查找指定的键
BTreeNode *search(BTreeNode *root, int key) {
    if (root == NULL) {
        return NULL;
    }

    int i = 0;
    // 找到第一个大于或等于 key 的键的索引
    while (i < root->n && key > root->keys[i]) {
        i++;
    }
    // 如果找到键
    if (i < root->n && key == root->keys[i]) {
        return root; // 返回包含该键的节点
    }
    // 如果是叶子节点且没有找到键
    if (root->isLeaf) {
        return NULL;
    }
    // 否则，在相应的子节点中递归查找
    return search(root->children[i], key);
}

```

遍历：

从最左边的孩子节点开始递归地打印最左边的孩子节点，然后对剩余的孩子和元素重复相同的过程，最后递归打印最右边的孩子节点。

```
// 遍历（中序遍历）
void traverse(BTreeNode *root) {
    if (root == NULL) {
        return;
    }

    int i;
    // 遍历当前节点的所有子节点（除了最后一个）
    for (i = 0; i < root->n; i++) {
        // 如果不是叶子节点，则递归遍历第 i 个子树
        if (!root->isLeaf) {
            traverse(root->children[i]);
        }
        printf("%d ", root->keys[i]); // 打印当前节点的第 i 个键
    }

    // 遍历最后一个子节点
    if (!root->isLeaf) {
        traverse(root->children[i]);
    }
}
```

插入节点：

```
// 当子节点已满时，将其分裂
void splitChild(BTreeNode *parent, int i, BTreeNode *child) {
    // 创建一个新的节点，用于存储 child 的后一半键和子节点
    BTreeNode *newChild = createNode(child->isLeaf);
    newChild->n = T - 1; // 新节点将包含 T-1 个键

    // 将 child 的后 T-1 个键复制到 newChild
    for (int j = 0; j < T - 1; j++) {
        newChild->keys[j] = child->keys[j + T];
    }

    // 如果 child 不是叶子节点，则将其后 T 个子节点复制到 newChild
    if (!child->isLeaf) {
        for (int j = 0; j < T; j++) {
            newChild->children[j] = child->children[j + T];
        }
    }

    child->n = T - 1; // child 节点现在只包含前 T-1 个键
}
```

```

// 将父节点中的子节点指针向后移动，为新子节点腾出空间
for (int j = parent->n; j >= i + 1; j--) {
    parent->children[j + 1] = parent->children[j];
}
parent->children[i + 1] = newChild; // 将新子节点链接到父节点

// 将父节点中的键向后移动，为从 child 提升的键腾出空间
for (int j = parent->n - 1; j >= i; j--) {
    parent->keys[j + 1] = parent->keys[j];
}
parent->keys[i] = child->keys[T - 1]; // 将 child 的中间键提升到父节点

parent->n = parent->n + 1; // 父节点中的键数量增加
}

// 将键插入到非满的节点中
void insertNonFull(BTreeNode *node, int key) {
    int i = node->n - 1; // 从当前节点的最右边开始

    // 如果是叶子节点
    if (node->isLeaf) {
        // 找到键应该插入的位置，并将大于 key 的键向右移动
        while (i >= 0 && key < node->keys[i]) {
            node->keys[i + 1] = node->keys[i];
            i--;
        }
        node->keys[i + 1] = key; // 插入键
        node->n = node->n + 1; // 键的数量增加
    } else { // 如果不是叶子节点
        // 找到键应该插入的子树
        while (i >= 0 && key < node->keys[i]) {
            i--;
        }
        i++; // 找到应该进入的子节点索引

        // 如果子节点已满，则分裂它
        if (node->children[i]->n == 2 * T - 1) {
            splitChild(node, i, node->children[i]);
            // 分裂后，如果 key 大于提升到父节点的键，则进入新的右子节点
            if (key > node->keys[i]) {
                i++;
            }
        }
        insertNonFull(node->children[i], key); // 递归插入
    }
}

// 插入键到B树中
void insert(BTree *tree, int key) {
    BTreeNode *root = tree->root;

```

```

// 如果根节点已满，则需要创建一个新的根节点并分裂旧的根节点
if (root->n == 2 * T - 1) {
    BTreeNode *newRoot = createNode(false); // 新根节点不是叶子节点
    // 旧根节点成为新根节点的第一个子节点
    newRoot->children[0] = root;
    tree->root = newRoot; // 更新树的根
    splitChild(newRoot, 0, root); // 分裂旧根节点
    insertNonFull(newRoot, key); // 在新根节点中插入键
} else {
    insertNonFull(root, key); // 根节点未满，直接插入
}
}

```

删除节点：

分类讨论如下：

- Case 1: 如果键在叶子节点中。
- Case 2: 如果键在非叶子节点中。
 - Case 2a: 如果左子节点有足够的键（至少 t 个），则用其前驱替换被删除的键。
 - Case 2b: 如果右子节点有足够的键（至少 t 个），则用其后继替换被删除的键。
 - Case 2c: 如果左右子节点键都不足，则将它们合并，并递归删除。
- Case 3: 如果键不在当前节点中，且当前节点是内部节点。
 - Case 3a: 如果子节点键不足（少于 t 个），从兄弟节点借键或合并。
 - Case 3b: 递归到相应的子节点。

```

// 辅助函数：查找键在节点中的索引
int findKeyIndex(BTreeNode *node, int key) {
    int idx = 0;
    while (idx < node->n && node->keys[idx] < key) {
        idx++;
    }
    return idx;
}

// 获取子树中的前驱（最大的键）
int findPredecessor(BTreeNode *node, int idx) {
    BTreeNode *curr = node->children[idx];
    while (!curr->isLeaf) {
        curr = curr->children[curr->n]; // 遍历到最右边的子节点
    }
    return curr->keys[curr->n - 1]; // 返回最右边的键
}

// 获取子树中的后继（最小的键）
int findSuccessor(BTreeNode *node, int idx) {
    BTreeNode *curr = node->children[idx + 1];

```

```

while (!curr->isLeaf) {
    curr = curr->children[0]; // 遍历到最左边的子节点
}
return curr->keys[0]; // 返回最左边的键
}

// 填充子节点：当子节点键的数量少于 T-1 时，从兄弟节点借键或合并
void fillChild(BTreeNode *node, int idx) {
    // 如果左兄弟有足够的键
    if (idx != 0 && node->children[idx - 1]->n >= T) {
        borrowFromLeft(node, idx);
    }
    // 如果右兄弟有足够的键
    else if (idx != node->n && node->children[idx + 1]->n >= T) {
        borrowFromRight(node, idx);
    }
    // 否则，合并子节点
    else {
        if (idx != node->n) {
            mergeChildren(node, idx);
        } else {
            mergeChildren(node, idx - 1);
        }
    }
}

// 从左兄弟借键
void borrowFromLeft(BTreeNode *node, int idx) {
    BTreeNode *child = node->children[idx];
    BTreeNode *leftSibling = node->children[idx - 1];

    // 将当前节点的一个键 (node->keys[idx-1]) 移动到 child 的最前面
    // 将 leftSibling 的最后一个键移动到当前节点的位置
    // 同时更新子节点指针
    for (int i = child->n - 1; i >= 0; i--) {
        child->keys[i + 1] = child->keys[i];
    }
    if (!child->isLeaf) {
        for (int i = child->n; i >= 0; i--) {
            child->children[i + 1] = child->children[i];
        }
    }

    child->keys[0] = node->keys[idx - 1];
    if (!child->isLeaf) {
        child->children[0] = leftSibling->children[leftSibling->n];
    }
    node->keys[idx - 1] = leftSibling->keys[leftSibling->n - 1];

    child->n++;
}

```

```

    leftSibling->n--;
}

// 从右兄弟借键
void borrowFromRight(BTreeNode *node, int idx) {
    BTreeNode *child = node->children[idx];
    BTreeNode *rightSibling = node->children[idx + 1];

    // 将当前节点的一个键 (node->keys[idx]) 移动到 child 的最后面
    // 将 rightSibling 的第一个键移动到当前节点的位置
    // 同时更新子节点指针
    child->keys[child->n] = node->keys[idx];
    if (!child->isLeaf) {
        child->children[child->n + 1] = rightSibling->children[0];
    }
    node->keys[idx] = rightSibling->keys[0];

    for (int i = 0; i < rightSibling->n - 1; i++) {
        rightSibling->keys[i] = rightSibling->keys[i + 1];
    }
    if (!rightSibling->isLeaf) {
        for (int i = 0; i < rightSibling->n; i++) {
            rightSibling->children[i] = rightSibling->children[i + 1];
        }
    }

    child->n++;
    rightSibling->n--;
}

// 合并子节点
void mergeChildren(BTreeNode *node, int idx) {
    BTreeNode *child = node->children[idx];
    BTreeNode *rightSibling = node->children[idx + 1];

    // 将父节点的键移动到 child 的末尾
    child->keys[T - 1] = node->keys[idx];

    // 将 rightSibling 的所有键移动到 child 的末尾
    for (int i = 0; i < rightSibling->n; i++) {
        child->keys[i + T] = rightSibling->keys[i];
    }

    // 如果不是叶子节点, 将 rightSibling 的所有子节点移动到 child
    if (!child->isLeaf) {
        for (int i = 0; i <= rightSibling->n; i++) {
            child->children[i + T] = rightSibling->children[i];
        }
    }
}

```

```

// 从父节点中删除键和相应的子节点指针
for (int i = idx + 1; i < node->n; i++) {
    node->keys[i - 1] = node->keys[i];
}
for (int i = idx + 2; i <= node->n; i++) {
    node->children[i - 1] = node->children[i];
}

child->n += rightSibling->n + 1; // 更新 child 的键数量
node->n--;                     // 父节点的键数量减少
free(rightSibling);           // 释放 rightSibling 节点
}

// 从节点中删除键
void deleteFromNode(BTreeNode *node, int key) {
    int idx = findKeyIndex(node, key);

    // Case 1: 键在当前节点中
    if (idx < node->n && node->keys[idx] == key) {
        // 如果是叶子节点，直接删除
        if (node->isLeaf) {
            for (int i = idx + 1; i < node->n; i++) {
                node->keys[i - 1] = node->keys[i];
            }
            node->n--;
        } else { // 如果是内部节点
            // Case 2a: 左子节点有足够的键
            if (node->children[idx]->n >= T) {
                int pred = findPredecessor(node, idx);
                node->keys[idx] = pred;
                deleteFromNode(node->children[idx], pred);
            }
            // Case 2b: 右子节点有足够的键
            else if (node->children[idx + 1]->n >= T) {
                int succ = findSuccessor(node, idx);
                node->keys[idx] = succ;
                deleteFromNode(node->children[idx + 1], succ);
            }
            // Case 2c: 两个子节点键都不足，合并
            else {
                mergeChildren(node, idx);
                deleteFromNode(node->children[idx], key);
            }
        }
    } else { // Case 3: 键不在当前节点中
        if (node->isLeaf) {
            printf("Key %d not found in the tree.\n", key);
            return;
        }
    }
}

```

```

    bool lastChild = (idx == node->n);

    // 如果子节点键不足，需要填充
    if (node->children[idx]->n < T) {
        fillChild(node, idx);
    }

    if (lastChild && idx > node->n) {
        // 如果原来是最后一个子节点，且父节点合并导致键数减少
        deleteFromNode(node->children[idx - 1], key);
    } else {
        deleteFromNode(node->children[idx], key);
    }
}

// 从B树中删除键
void delete(BTree *tree, int key) {
    if (tree->root == NULL) {
        printf("Tree is empty.\n");
        return;
    }

    deleteFromNode(tree->root, key);

    // 如果根节点变为空（除了一个子节点），则将根节点设置为它的子节点
    if (tree->root->n == 0) {
        BTreeNode *oldRoot = tree->root;
        if (oldRoot->isLeaf) {
            // 如果根节点是叶子节点且为空，树为空
            tree->root = NULL;
        } else {
            // 如果根节点是内部节点且为空，将其子节点作为新根
            tree->root = oldRoot->children[0];
        }
        free(oldRoot); // 释放旧根节点
    }
}

```

B+ Tree

（这部分貌似不是很重要，知道有这个概念即可）

B+ 树可以看作是 B 树的一种变形，在实现文件索引结构方面比 B 树使用得更普遍。

特点：

- 所有的叶子结点中包含了全部关键字的信息，及指向含有这些关键字记录的指针；

- 所有的非终端结点可以看成是索引；
- 结点中仅含有其子树根结点中最大（或最小）关键字；
- 通常在B+树上有两个头指针，一个指向根结点，另一个指向关键字最小的叶子结点。

遍历

查找：

在搜索过程中，若非终端结点上的关键字等于给定值，搜索并不停止，而是继续沿右指针向下，一直查到叶结点上的这个关键字。

在B+树，不管查找成功与否，每次查找都是走了一条从根到叶子结点的路径。

B+树的查找特点：

1. 在B+树上，既可以进行缩小范围的查找，也可以进行顺序查找。
2. 在进行缩小范围的查找时，不管成功与否，都必须查到叶子结点才能结束。
3. 若在结点内查找时，给定值 $\leq K_i$ ，则应继续在 A_i 所指子树中进行查找。

插入：

插入操作可能导致节点分裂，并且需要将分裂后的中间键提升到父节点。

B+树的键提升与B树略有不同，分裂的叶子节点中的最小键会提升到父节点，但键本身仍然保留在叶子节点。

删除：

删除操作的复杂性与B树相似，也需要处理借用（redistribution）和合并（merge）的情况，以维护B+树的最小度属性。